

Four points, and a line that cannot be drawn

Run live at the front. One neuron — the one you trained on Tuesday — meets four points it cannot separate. Then a hidden layer does what the neuron could not, previewing Tuesday.

Live demonstration

No hand-in

Pen and paper

Sets up session 13

Before you start

1 · four points

2 · one neuron tries

3 · the history

4 · the fix, previewed

5 · what the hidden units did

Before you start

This notebook is run at the front by the instructor. Follow along with pen and paper; you may run it yourself afterwards.



By the end of this session you can

- Say why no single straight line separates XOR.
- Recognise a loss stuck at 0.693 as a model that has learned nothing.
- Explain what a hidden layer changes, before you compute one in session 13.

STEP 1

four points

1

The XOR truth table as a picture. **On your paper:** try to draw one straight line with both 1s on one side and both 0s on the other.

in the notebook

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = np.array([0., 1., 1., 0.])

plt.scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm", s=200, edgecolor="k")
plt.xlim(-0.3, 1.3); plt.ylim(-0.3, 1.3)
plt.xlabel("x1"); plt.ylabel("x2"); plt.title("XOR")
plt.show()
```

STEP 2

one neuron tries

2

The exact training loop from Lab 8, started from small random weights so the failure cannot be blamed on starting at zero.

in the notebook

PYTHON

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

rng = np.random.default_rng(12)
w, b, lr = rng.normal(0, 0.5, 2), 0.0, 0.5
for epoch in range(5000):
    p = sigmoid(X @ w + b)
    dz = p - y
    w -= lr * X.T @ dz / 4
    b -= lr * dz.mean()

p = sigmoid(X @ w + b)
loss = -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
print("probabilities:", np.round(p, 3))
print("loss:", round(loss, 4), "    log 2 =", round(np.log(2), 4))
```

what it should print

OUTPUT

```
probabilities: [0.5 0.5 0.5 0.5]
loss: 0.6931    log 2 = 0.6931
```



● **Five thousand epochs, and the neuron says 0.5 to everything.** The loss is $-\log 2$ — the number from Lab 8 that means *learned nothing*. It is not a bug and more training will not help: the best a single line can do on these four points is to refuse to decide.

STEP 3

the history

3

Rosenblatt's perceptron (1958) was a single neuron like this one. Minsky and Papert (1969) proved it could not represent XOR, and the proof helped bring on the first AI winter — the slide from session 3.

STEP 4

the fix, previewed

4

Add a hidden layer of three sigmoid units between the inputs and the output. The weights below were set by hand, not learned — Tuesday you compute this exact forward pass yourself, and session 14 shows how weights like these are found.

in the notebook

PYTHON

```
W1 = np.array([[ 4.0,  4.0, -4.0],
               [ 4.0, -4.0,  4.0]])
b1 = np.array([-2.0, 2.0, 2.0])
W2 = np.array([[6.0], [-6.0], [-6.0]])
b2 = np.array([3.0])

A1 = sigmoid(X @ W1 + b1)
out = sigmoid(A1 @ W2 + b2).ravel()
print("2-3-1 network:", np.round(out, 3), " targets:", y)
```

what it should print

OUTPUT

```
2-3-1 network: [0.001 0.83  0.83  0.17 ] targets: [0. 1. 1. 0.]
```

what the hidden units did

Each hidden unit draws its **own** line. The output unit then combines three yes/no answers — and a combination of lines is not a line.

in the notebook

PYTHON

```
xx, yy = np.meshgrid(np.linspace(-0.3, 1.3, 120), np.linspace(-0.3, 1.3, 120))
grid = np.c_[xx.ravel(), yy.ravel()]
H = sigmoid(grid @ W1 + b1)
fig, ax = plt.subplots(1, 4, figsize=(14, 3.2))
for j in range(3):
    ax[j].contourf(xx, yy, H[:, j].reshape(xx.shape), levels=20, cmap="viridis")
    ax[j].scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm", edgecolor="w");
ax[j].set_title(f"hidden unit {j}")
O = sigmoid(H @ W2 + b2).reshape(xx.shape)
ax[3].contourf(xx, yy, O, levels=20, cmap="coolwarm"); ax[3].scatter(X[:, 0], X[:, 1], c=y,
cmap="coolwarm", edgecolor="k")
ax[3].set_title("output")
for a in ax: a.set_xlabel("x1"); a.set_ylabel("x2")
plt.tight_layout(); plt.show()
```

Discuss with your neighbour

1. Why does Step 2's neuron settle on 0.5 rather than, say, getting three of the four points right?
2. Lab 5 proved two linear layers collapse into one. Which ingredient in Step 4 stops the collapse?
3. Count the parameters in Step 4. You should get 9 weights and 4 biases — the network from slide 21.